

Web development

Labo 21
Jari Rhellam

1. Globale variabelen

Opdracht: demo storage

Ik heb de map **Demo 1 globale variabelen** geopend en de pagina getest.

Observaties:

- Bij herladen van de pagina (**F5**) gaat de teller terug naar 0.
- Bij sluiten van het tabblad en opnieuw openen is de teller ook weg.
- Twee verschillende tabbladen gebruiken **niet** dezelfde teller (elke tab heeft zijn eigen globale variabele).

Conclusie: Globale variabelen werken alleen zolang de pagina in hetzelfde tabblad open blijft. Zodra je herlaadt of het tabblad sluit, zijn ze verloren.

Code:

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <meta charset="UTF-8">
  <title>Demo 1 - Globale variabelen</title>
  <script src="script/code.js" defer></script>
</head>
<body>
  <h1>Globale variabelen demo</h1>
  <p>Klik op de knop om de teller te verhogen.</p>
  <button id="btnVerhoog">Verhoog teller</button>
  <p id="txtResult">(nog niet geklikt)</p>
</body>
</html>
```

```
let teller = 0; // globale variabele
```

```
const setup = () => {
  let btnVerhoog = document.getElementById("btnVerhoog");
  btnVerhoog.addEventListener("click", verhoog);
};
```

```
const verhoog = () => {
  let txtResult = document.getElementById("txtResult");
  teller++;
  txtResult.innerHTML = "De waarde van de teller is " + teller;
};

window.addEventListener("load", setup);
```

2. Gegevens over meerdere pagina's heen

Demo 2 – Session Storage

Observaties:

- De teller blijft behouden bij herladen van de pagina (**F5**).
- Zodra je het tabblad volledig sluit, is de teller weg.
- Twee verschillende tabbladen hebben elk hun eigen teller.

Conclusie: Session Storage is gekoppeld aan één sessie (één tabblad). Het verdwijnt wanneer het tabblad gesloten wordt.

Code:

script/code.js

```
const setup = () => {
  let btnVerhoog = document.getElementById("btnVerhoog");
  btnVerhoog.addEventListener("click", verhoog);
};

const verhoog = () => {
  let txtResult = document.getElementById("txtResult");

  if (typeof(Storage) === "undefined") {
    txtResult.innerHTML = "Sorry, uw browser ondersteunt geen web storage...";
    return;
  }

  let clickCount = sessionStorage.getItem("demo.clickCount");
  clickCount = clickCount ? parseInt(clickCount) + 1 : 1;

  sessionStorage.setItem("demo.clickCount", clickCount.toString());
  txtResult.innerHTML = "De waarde van de teller is " + clickCount;
};

window.addEventListener("load", setup);
```

Demo 3 – Local Storage

Observaties:

- De teller blijft behouden bij herladen van de pagina.
- De teller blijft behouden nadat het tabblad of de browser volledig gesloten is.
- Alle tabbladen (zelfde domein) delen dezelfde teller.

Conclusie: Local Storage is persistent (blijft bestaan tot je het manueel verwijdert of de browser data wist).

Code: (bijna identiek aan session storage, alleen localStorage i.p.v. sessionStorage)

```
const verhoog = () => {
  let txtResult = document.getElementById("txtResult");

  if (typeof(Storage) === "undefined") {
    txtResult.innerHTML = "Sorry, uw browser ondersteunt geen web storage..";
    return;
  }

  let clickCount = localStorage.getItem("demo.clickCount");
  clickCount = clickCount ? parseInt(clickCount) + 1 : 1;

  localStorage.setItem("demo.clickCount", clickCount.toString());
  txtResult.innerHTML = "De waarde van de teller is " + clickCount;
};
```

3. Demo JSON – Gegevens opslaan als object

Opdracht: Settings (leeftijd, gewicht, budget) opslaan via JSON.stringify() en terug inlezen met JSON.parse().

Observaties:

- Na het invullen en klikken op **Save settings** blijven de waarden behouden na herladen.
- Via **Load settings** worden de opgeslagen waarden terug in de velden gezet.
- De data wordt als één JSON-string opgeslagen in localStorage onder de key "VIVES.be.settingsPerson".

Code:

index.html (belangrijkste deel)

```

<label>Leeftijd: <input type="number" id="txtAge"></label><br><br>
<label>Gewicht (kg): <input type="number" id="txtWeight"></label><br><br>
<label>Dagelijks budget (€): <input type="number" id="txtBudget"></label><br><br>

<button id="btnSave">Save settings</button>
<button id="btnShow">Load settings</button>

```

```

const saveSettings = () => {
  let settings = {
    age: parseInt(document.getElementById("txtAge").value) || 0,
    weight: parseInt(document.getElementById("txtWeight").value) || 0,
    budget: parseInt(document.getElementById("txtBudget").value) || 0
  };

  let settingsJSON = JSON.stringify(settings);
  localStorage.setItem("VIVES.be.settingsPerson", settingsJSON);
  alert("Settings succesvol opgeslagen!");
};

const showSettings = () => {
  let settingsJSON = localStorage.getItem("VIVES.be.settingsPerson");
  let settings = { age: 0, weight: 0, budget: 0 };

  if (settingsJSON) {
    settings = JSON.parse(settingsJSON);
  }

  document.getElementById("txtAge").value = settings.age;
  document.getElementById("txtWeight").value = settings.weight;
  document.getElementById("txtBudget").value = settings.budget;
};

```

4. Opdracht: ColorPicker Pro

Doel: De Color Picker uitbreiden zodat de opgeslagen kleuren **blijvend** bewaard blijven in localStorage, ook na sluiten van de browser.

Functionaliteiten toegevoegd:

- Kleuren opslaan via **Save color** → blijft bestaan na herladen/browser sluiten.
- Opgeslagen kleuren tonen bij het laden van de pagina.
- Klikken op een opgeslagen kleur → herstelt die kleur in de sliders.
- Delete knop (×) per kleur.
- **Clear all saved colors** knop met bevestiging.

Code:

scripts/colorPickerPro.js (belangrijkste bestand)

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <meta charset="UTF-8">
  <link rel="stylesheet" href="colorPicker.css">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Color Picker Pro</title>
</head>
<body>
<div class="colorPicker">
  <div class="components">
    <div class="component">
      <span>0 <label for="sldrood"></label><input id="sldrood" type="range"
class="slider" value="128" min="0" max="255"> 255</span>
      <span>Rood</span>
      <span id="rood">128</span>
    </div>
    <div class="component">
      <span>0 <label for="sldgroen"></label><input id="sldgroen" type="range"
class="slider" value="128" min="0" max="255"> 255</span>
      <span>Groen</span>
      <span id="groen">128</span>
    </div>
    <div class="component">
      <span>0 <label for="sldblauw"></label><input id="sldblauw" type="range"
class="slider" value="128" min="0" max="255"> 255</span>
      <span>Blauw</span>
      <span id="blauw">128</span>
    </div>
  </div>
  <div class="swatch" id="swatch"></div>
  <input type="button" id="btnSave" value="Save color">
  <input type="button" id="btnClear" value="Clear all saved colors">
</div>

<div id="divColor"></div>

<script src="scripts/colorPickerPro.js"></script>
</body>
</html>
```

```
.slider { vertical-align: middle; }
.component span:nth-of-type(2) {
  display: inline-block;
```

```

width: 3em;
text-align: right;
}
.swatch {
display: inline-block;
border: solid 1px black;
border-radius: 5px;
margin-left: 1em;
height: 4em;
width: 5em;
}
.components { display: inline-block; height: 100%; }
.colorPicker {
display: inline-block;
border: solid 5px lightgrey;
border-radius: 10px;
padding: 10px;
}
#divColor div {
margin: 5px;
}

```

```

const setup = () => {
const sliders = document.getElementsByClassName("slider");
for (let i = 0; i < sliders.length; i++) {
sliders[i].addEventListener("input", update);
sliders[i].addEventListener("change", update);
}

document.getElementById("btnSave").addEventListener("click", saveColor);
document.getElementById("btnClear").addEventListener("click", clearAllColors);

loadSavedColors(); // ← nieuw: laad opgeslagen kleuren bij starten
update();
};

const update = () => {
const valueRed = document.getElementById("sldrood").value;
const valueGreen = document.getElementById("sldgroen").value;
const valueBlue = document.getElementById("sldblauw").value;

document.getElementById("rood").innerHTML = valueRed;
document.getElementById("groen").innerHTML = valueGreen;
document.getElementById("blauw").innerHTML = valueBlue;
}

```

```

const swatch = document.getElementById("swatch");
swatch.style.backgroundColor = `rgb(${valueRed}, ${valueGreen}, ${valueBlue})`;
};

const saveColor = () => {
  const r = document.getElementById("sldrood").value;
  const g = document.getElementById("sldgroen").value;
  const b = document.getElementById("sldblauw").value;

  const colorKey = `color_${Date.now()}`; // unieke key

  const colorData = {
    r: r,
    g: g,
    b: b
  };

  localStorage.setItem(colorKey, JSON.stringify(colorData));

  addColorToDOM(colorKey, r, g, b);
};

const addColorToDOM = (key, r, g, b) => {
  const divColors = document.getElementById("divColor");

  const obj = document.createElement("div");
  obj.style.backgroundColor = `rgb(${r}, ${g}, ${b})`;
  obj.style.position = "relative";
  obj.style.width = "70px";
  obj.style.height = "70px";
  obj.style.display = "inline-block";
  obj.dataset.key = key; // om later te verwijderen

  obj.addEventListener("click", restoreColor);

  // delete button
  const btnDelete = document.createElement("button");
  btnDelete.textContent = "x";
  btnDelete.style.position = "absolute";
  btnDelete.style.top = "2px";
  btnDelete.style.right = "2px";

  btnDelete.addEventListener("click", (event) => {
    event.stopPropagation();
    deleteColor(key, obj);
  });
};

```

```

obj.appendChild(btnDelete);
divColors.appendChild(obj);
};

const restoreColor = (event) => {
  const obj = event.currentTarget;
  const r = obj.style.backgroundColor.match(/\d+/g)[0];
  const g = obj.style.backgroundColor.match(/\d+/g)[1];
  const b = obj.style.backgroundColor.match(/\d+/g)[2];

  document.getElementById("sldrood").value = r;
  document.getElementById("sldgroen").value = g;
  document.getElementById("sldblauw").value = b;
  update();
};

const deleteColor = (key, element) => {
  localStorage.removeItem(key);
  element.remove();
};

const clearAllColors = () => {
  if (confirm("Weet je zeker dat je alle opgeslagen kleuren wilt verwijderen?")) {
    const divColors = document.getElementById("divColor");
    divColors.innerHTML = "";

    // verwijder alle keys die met "color_" beginnen
    Object.keys(localStorage).forEach(key => {
      if (key.startsWith("color_")) {
        localStorage.removeItem(key);
      }
    });
  }
};

const loadSavedColors = () => {
  Object.keys(localStorage).forEach(key => {
    if (key.startsWith("color_")) {
      const data = JSON.parse(localStorage.getItem(key));
      addColorToDOM(key, data.r, data.g, data.b);
    }
  });
};

window.addEventListener("load", setup);

```

